

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA5113

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I DINDU PUJITHA(192373037) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D PUJITHA

Annexure A

Experiments

- 1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.**

Answer:

Code:

```
import hashlib
import time

messages = [
    "Hello World",
    "Cryptography",
    "Secure File Transfer",
    "Python Programming",
    "Information Security"
]

print("MD5 vs SHA-256 Comparison")
print("-" * 80)

for message in messages:
    # MD5
    start = time.perf_counter()
    md5_hash = hashlib.md5(message.encode()).hexdigest()
    md5_time = time.perf_counter() - start

    # SHA-256
    start = time.perf_counter()
```

```

sha256_hash = hashlib.sha256(message.encode()).hexdigest()

sha256_time = time.perf_counter() - start


print("\nMessage:", message)
print("MD5   :", md5_hash)
print("SHA-256:", sha256_hash)
print("MD5 Time   :", md5_time)
print("SHA-256 Time :", sha256_time)


print("\nAnalysis:")
print("MD5 produces a 128-bit (32 hexadecimal character) hash.")
print("SHA-256 produces a 256-bit (64 hexadecimal character) hash.")
print("MD5 is generally faster but is cryptographically broken.")
print("SHA-256 provides significantly stronger collision resistance.")

```

Output:

The screenshot shows a code editor with a Python file named 'main.py'. The code imports 'hashlib' and 'time', defines a list of messages, and then iterates over each message to calculate MD5 and SHA-256 hashes and their execution times. The output window on the right displays the results for two messages: 'Python Programming' and 'Information Security'. The output shows the message, the MD5 hash, the SHA-256 hash, the MD5 execution time, and the SHA-256 execution time. The analysis section at the bottom of the output window states that MD5 produces a 128-bit (32 hexadecimal character) hash, SHA-256 produces a 256-bit (64 hexadecimal character) hash, MD5 is generally faster but is cryptographically broken, and SHA-256 provides significantly stronger collision resistance. The code execution was successful.

```

main.py Visualize Run
1 import hashlib
2 import time
3 messages = [
4     "Hello World",
5     "Cryptography",
6     "Secure File Transfer",
7     "Python Programming",
8     "Information Security"
9 ]
10 print("MD5 vs SHA-256 Comparison")
11 print("-" * 80)
12 for message in messages:
13     md5_hash = hashlib.md5(message.encode()).hexdigest()
14     md5_time = time.perf_counter() - start
15     start = time.perf_counter()
16     sha256_hash = hashlib.sha256(message.encode()).hexdigest()
17     sha256_time = time.perf_counter() - start
18     print("\nMessage:", message)
19     print("MD5   :", md5_hash)
20     print("SHA-256:", sha256_hash)
21     print("MD5 Time   :", md5_time)
22     print("SHA-256 Time :", sha256_time)

```

Output

```

Message: Python Programming
MD5   : de6ae90465e75b350ed18a2024ca16ef
SHA-256: d15647d90cc9f8e341146110bf5058773171688b237d2f9b5704399a0b17c8bd
MD5 Time   : 1.4499528333544731e-06
SHA-256 Time : 9.700306691229343e-07

Message: Information Security
MD5   : 0889f4cee1d9f42ae2c1daf686377de0
SHA-256: 620828d6c932bb62090152cc230e9bf082d541961b9397a1bf7ce851c06e77ad
MD5 Time   : 1.1999509297311306e-06
SHA-256 Time : 8.699717000126839e-07

Analysis:
MD5 produces a 128-bit (32 hexadecimal character) hash.
SHA-256 produces a 256-bit (64 hexadecimal character) hash.
MD5 is generally faster but is cryptographically broken.
SHA-256 provides significantly stronger collision resistance.

=== Code Execution Successful ===

```

2. **Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.**

Answer:

Code:

```
import hashlib

p = 61
q = 53

n = p * q
phi = (p - 1) * (q - 1)

e = 17
d = pow(e, -1, phi)

print("RSA Keys Generated")
print("Public Key :", (e, n))
print("Private Key:", (d, n))

message = "Hello, this is a secure message."

print("\nOriginal Message:")
print(message)

hash_value = hashlib.sha256(message.encode()).hexdigest()
hash_integer = int(hash_value, 16) % n

print("\nSHA-256 Hash:")
print(hash_value)
```

```
signature = pow(hash_integer, d, n)

print("\nDigital Signature:")
print(signature)

verification = pow(signature, e, n)

if verification == hash_integer:
    print("\nSignature Verification: SUCCESS")
    print("The message is authentic and has not been modified.")
else:
    print("\nSignature Verification: FAILED")

modified_message = "Hello, this message was modified."

modified_hash = hashlib.sha256(
    modified_message.encode()
).hexdigest()

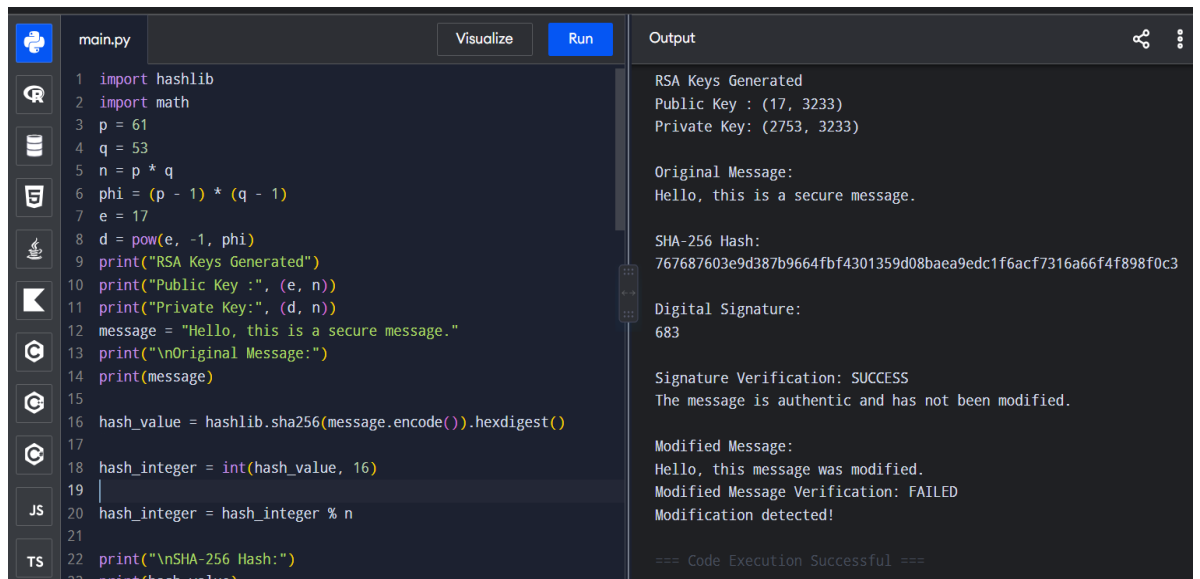
modified_hash_integer = int(modified_hash, 16) % n

print("\nModified Message:")
print(modified_message)

verification = pow(signature, e, n)

if verification == modified_hash_integer:
    print("Modified Message Verification: SUCCESS")
else:
    print("Modified Message Verification: FAILED")
    print("Modification detected!")
```

Output:



```
main.py Visualize Run Output
1 import hashlib
2 import math
3 p = 61
4 q = 53
5 n = p * q
6 phi = (p - 1) * (q - 1)
7 e = 17
8 d = pow(e, -1, phi)
9 print("RSA Keys Generated")
10 print("Public Key :", (e, n))
11 print("Private Key:", (d, n))
12 message = "Hello, this is a secure message."
13 print("\nOriginal Message:")
14 print(message)
15
16 hash_value = hashlib.sha256(message.encode()).hexdigest()
17
18 hash_integer = int(hash_value, 16)
19
20 hash_integer = hash_integer % n
21
22 print("\nSHA-256 Hash:")
23 print(hash_value)
```

RSA Keys Generated
Public Key : (17, 3233)
Private Key: (2753, 3233)

Original Message:
Hello, this is a secure message.

SHA-256 Hash:
767687603e9d387b9664fbf4301359d08baea9edc1f6acf7316a66f4f898f0c3

Digital Signature:
683

Signature Verification: SUCCESS
The message is authentic and has not been modified.

Modified Message:
Hello, this message was modified.
Modified Message Verification: FAILED
Modification detected!

=== Code Execution Successful ===

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

Answer:

Code:

```
import hashlib
import hmac

message = b"Hello, this is a secure message."
key = b"secretkey123"

simple_hash = hashlib.sha256(message).hexdigest()

hmac_value = hmac.new(
    key,
    message,
    hashlib.sha256
).hexdigest()

print("Original Message:")
print(message.decode())

print("\nSimple SHA-256 Hash:")
print(simple_hash)

print("\nHMAC-SHA256:")
print(hmac_value)

received_message = b"Hello, this is a secure message."
```

```
received_hmac = hmac.new(
    key,
    received_message,
    hashlib.sha256
).hexdigest()

if hmac.compare_digest(hmac_value, received_hmac):
    print("\nHMAC Verification: SUCCESS")
    print("Message is authentic and unchanged.")
else:
    print("\nHMAC Verification: FAILED")

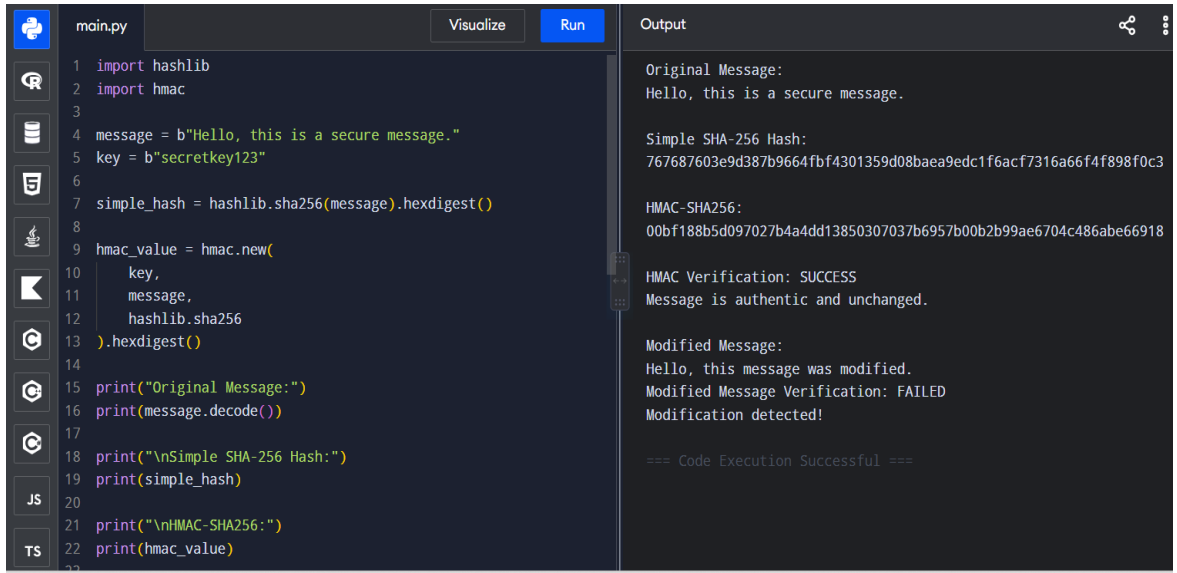
modified_message = b"Hello, this message was modified."

modified_hmac = hmac.new(
    key,
    modified_message,
    hashlib.sha256
).hexdigest()

print("\nModified Message:")
print(modified_message.decode())

if hmac.compare_digest(hmac_value, modified_hmac):
    print("Modified Message Verification: SUCCESS")
else:
    print("Modified Message Verification: FAILED")
    print("Modification detected!")
```

Output:



The image shows a code editor interface with a file named `main.py`. The code implements HMAC and SHA-256 hashing for a message. The output panel on the right shows the results of the code execution.

```
1 import hashlib
2 import hmac
3
4 message = b"Hello, this is a secure message."
5 key = b"secretkey123"
6
7 simple_hash = hashlib.sha256(message).hexdigest()
8
9 hmac_value = hmac.new(
10     key,
11     message,
12     hashlib.sha256
13 ).hexdigest()
14
15 print("Original Message:")
16 print(message.decode())
17
18 print("\nSimple SHA-256 Hash:")
19 print(simple_hash)
20
21 print("\nHMAC-SHA256:")
22 print(hmac_value)
```

Output:

Original Message:
Hello, this is a secure message.

Simple SHA-256 Hash:
767687603e9d387b9664fbf4301359d08baea9edc1f6acf7316a66f4f898f0c3

HMAC-SHA256:
00bf188b5d097027b4a4dd13850307037b6957b00b2b99ae6704c486abe66918

HMAC Verification: SUCCESS
Message is authentic and unchanged.

Modified Message:
Hello, this message was modified.
Modified Message Verification: FAILED
Modification detected!

=== Code Execution Successful ===

4. **Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.**

Answer:

Code:

```
import time
import secrets

class AuthenticationServer:
    def __init__(self):
        self.users = {
            "alice": "password123"
        }

    def authenticate(self, username, password):
        if username in self.users and self.users[username] == password:
            print("Authentication Server: User authenticated.")
            return True
        else:
            print("Authentication Server: Authentication failed.")
            return False

class TicketGrantingServer:
    def generate_ticket(self, username):
        ticket = {
            "username": username,
            "ticket_id": secrets.token_hex(8),
            "timestamp": time.time(),
            "lifetime": 60
        }
```

```
print("Ticket Granting Server: Ticket generated.")
return ticket
```

```
class ApplicationServer:
```

```
    def __init__(self):
```

```
        self.used_tickets = set()
```

```
    def validate_ticket(self, ticket):
```

```
        current_time = time.time()
```

```
        if ticket["ticket_id"] in self.used_tickets:
```

```
            print("Server: REPLAY ATTACK DETECTED!")
```

```
            return False
```

```
        if current_time - ticket["timestamp"] > ticket["lifetime"]:
```

```
            print("Server: Ticket expired!")
```

```
            return False
```

```
        self.used_tickets.add(ticket["ticket_id"])
```

```
        print("Server: Ticket is valid.")
```

```
        print("Server: Access granted to", ticket["username"])
```

```
        return True
```

```
AS = AuthenticationServer()
```

```
TGS = TicketGrantingServer()
```

```
SERVER = ApplicationServer()
```

```
username = "alice"
```

```
password = "password123"
```

```

print("Client requesting authentication...\n")

if AS.authenticate(username, password):

    ticket = TGS.generate_ticket(username)

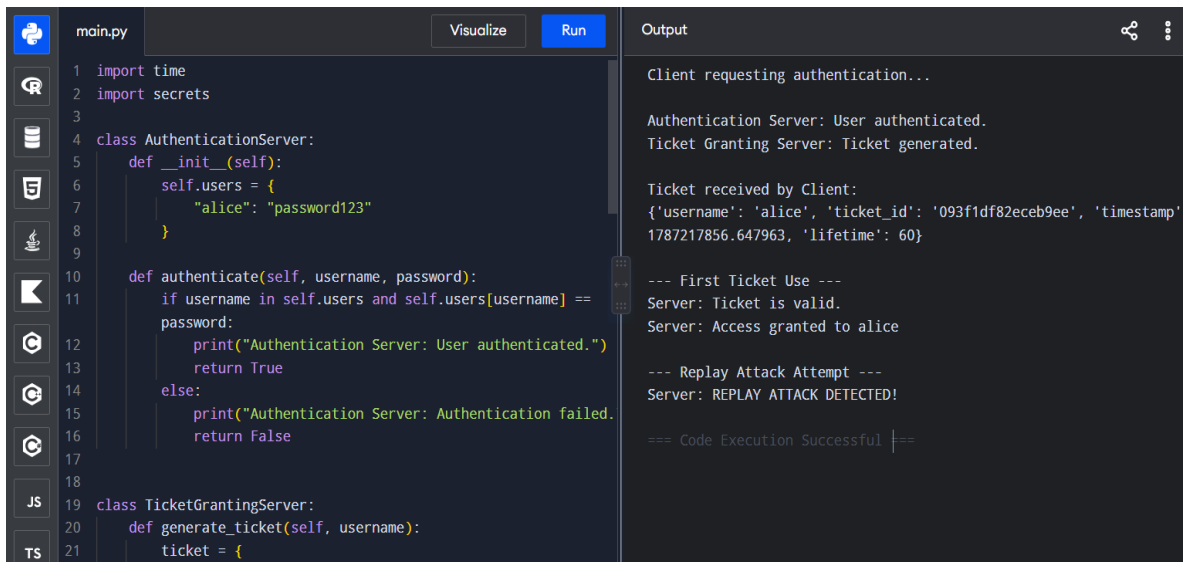
    print("\nTicket received by Client:")
    print(ticket)

    print("\n--- First Ticket Use ---")
    SERVER.validate_ticket(ticket)

    print("\n--- Replay Attack Attempt ---")
    SERVER.validate_ticket(ticket)

```

Output:



The screenshot shows a code editor with a file named `main.py`. The code defines two classes: `AuthenticationServer` and `TicketGrantingServer`. The `AuthenticationServer` class has a `__init__` method that initializes a dictionary of users with 'alice' having password 'password123'. It also has an `authenticate` method that checks the username and password. The `TicketGrantingServer` class has a `generate_ticket` method that generates a ticket for a given username. The code is executed, and the output is displayed on the right side of the editor.

```

main.py Visualize Run
1 import time
2 import secrets
3
4 class AuthenticationServer:
5     def __init__(self):
6         self.users = {
7             "alice": "password123"
8         }
9
10    def authenticate(self, username, password):
11        if username in self.users and self.users[username] ==
12            password:
13            print("Authentication Server: User authenticated.")
14            return True
15        else:
16            print("Authentication Server: Authentication failed.")
17            return False
18
19 class TicketGrantingServer:
20     def generate_ticket(self, username):
21         ticket = {

```

Output

```

Client requesting authentication...

Authentication Server: User authenticated.
Ticket Granting Server: Ticket generated.

Ticket received by Client:
{'username': 'alice', 'ticket_id': '093f1df82eceb9ee', 'timestamp'
1787217856.647963, 'lifetime': 60}

--- First Ticket Use ---
Server: Ticket is valid.
Server: Access granted to alice

--- Replay Attack Attempt ---
Server: REPLAY ATTACK DETECTED!

=== Code Execution Successful ===

```

5. **Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.**

Answer:

Code:

```
import hashlib
import math
import secrets

p = 467
g = 2

private_key = secrets.randbelow(p - 2) + 1
public_key = pow(g, private_key, p)

print("ElGamal Digital Signature")
print("-----")

print("Private Key:", private_key)
print("Public Key :", public_key)

message = "Hello, this is a secure message."

print("\nOriginal Message:")
print(message)

hash_value = int.from_bytes(
    hashlib.sha256(message.encode()).digest(),
    "big"
) % (p - 1)
```

```

print("\nSHA-256 Hash:")
print(hash_value)

while True:
    k = secrets.randbelow(p - 2) + 2
    if math.gcd(k, p - 1) == 1:
        break

r = pow(g, k, p)

k_inverse = pow(k, -1, p - 1)

s = ((hash_value - private_key * r) * k_inverse) % (p - 1)

print("\nDigital Signature:")
print("r =", r)
print("s =", s)

left = (pow(public_key, r, p) * pow(r, s, p)) % p
right = pow(g, hash_value, p)

if left == right:
    print("\nSignature Verification: SUCCESS")
    print("The message is authentic and has not been modified.")
else:
    print("\nSignature Verification: FAILED")

modified_message = "Hello, this message was modified."

modified_hash = int.from_bytes(
    hashlib.sha256(modified_message.encode()).digest(),
    "big"
) % (p - 1)

```

```

left = (pow(public_key, r, p) * pow(r, s, p)) % p
right = pow(g, modified_hash, p)

print("\nModified Message:")
print(modified_message)

if left == right:
    print("Modified Message Verification: SUCCESS")
else:
    print("Modified Message Verification: FAILED")
    print("Modification detected!")

```

Output:

The screenshot shows a code editor with a Python file named 'main.py'. The code implements ElGamal digital signature and verification. It generates a private key (228) and a public key (394). The original message is 'Hello, this is a secure message.' The SHA-256 hash of the message is 349. The digital signature is generated with r=99 and s=369. The signature verification is successful, confirming the message is authentic. The code then modifies the message to 'Hello, this message was modified.' and shows that the verification fails, detecting the modification.

```

1 import hashlib
2 import math
3 import secrets
4
5 p = 467
6 g = 2
7
8 private_key = secrets.randbelow(p - 2) + 1
9 public_key = pow(g, private_key, p)
10
11 print("ElGamal Digital Signature")
12 print("-----")
13
14 print("Private Key:", private_key)
15 print("Public Key :", public_key)
16
17 message = "Hello, this is a secure message."
18
19 print("\nOriginal Message:")
20 print(message)
21
22 hash_value = int.from_bytes(

```

Output

```

Private Key: 228
Public Key : 394

Original Message:
Hello, this is a secure message.

SHA-256 Hash:
349

Digital Signature:
r = 99
s = 369

Signature Verification: SUCCESS
The message is authentic and has not been modified.

Modified Message:
Hello, this message was modified.
Modified Message Verification: FAILED
Modification detected!

=== Code Execution Successful ===

```